

Vježba 15

Pytorch - Uvod i pokretanje gotove CNN mreže



Sadržaj

- Pytorch instalacija
- Tensor - osnovni objekt u Pytorchu
- PyTorch Tensor vs Numpy array
- Dataset i DataLoader
- Transformacije
- Učitavanje gotove CNN mreže – ResNet18
- Loss i Optimizer
- Trening i Validacijska petlja
- Spremanje modela na disk, učitavanje modela s diska i inferencija
- Tensorboard u PyTorchu

Pytorch instalacija

Pytorch instalacija

Instalacija PyTorch

PyTorch se može instalirati u nekoliko varijanti, ovisno o hardveru:

- **CPU verzija**
 - radi na svakom računalu
 - jednostavna instalacija
 - sporija za treniranje većih neuronskih mreža
- **GPU (CUDA) verzija**
 - koristi NVIDIA grafičku karticu
 - značajno ubrzava treniranje
 - zahtijeva kompatibilnu CUDA verziju

Preporuka:

Ako imate NVIDIA GPU, koristite CUDA verziju. U suprotnom, CPU verzija je sasvim dovoljna za ove vježbe.

Službena stranica za instalaciju:

<https://pytorch.org/get-started/locally/>

Pytorch instalacija

Instalacija PyTorch – primjeri i provjera

Instalacija (CPU verzija – primjer za Windows / pip):

```
pip3 install torch torchvision
```

Instalacija (CUDA verzija – NVIDIA GPU):

```
pip3 install torch torchvision --index-url https://download.pytorch.org/whl/cu126
```



Napomena: CUDA verzija mora biti kompatibilna s vašom grafičkom karticom i driverima.
Točnu naredbu uvijek provjerite na službenoj PyTorch stranici.

Provjera instalacije:

```
import torch  
torch.cuda.is_available()
```

True → PyTorch vidi GPU i CUDA je ispravno instalirana

False → koristi se CPU verzija

Tensor - osnovni objekt u PyTorchu

Tensor - osnovni objekt u PyTorchu

Tensor je osnovna struktura podataka u PyTorchu.

Predstavlja višedimenzionalno numeričko polje (skalar, vektor, matrica, tenzor).

- vrlo sličan **NumPy arrayu**
- može se izvršavati na **CPU-u** ili **GPU-u**
- podržava **automatsko računanje gradijenata**

Tensor može imati:

- oblik (shape)
- tip podataka (dtype)
- uređaj (device)
- informaciju o gradijentima (requires_grad)



Svi podaci, ulazi, izlazi i parametri modela u PyTorchu su Tensori.

Tensor - osnovni objekt u PyTorchu

Stvaranje Tensora u PyTorchu

Tensor se može stvoriti na više načina, ovisno o potrebi.

Ručno stvaranje:

```
import torch  
x = torch.tensor([1, 2, 3])
```

Tensor s nulama ili jedinicama:

```
torch.zeros(3, 4)  
torch.ones(2, 2)
```

Nasumični Tensor

```
torch.rand(3, 3)
```

Definiranje tipa podataka (dtype)

```
torch.tensor([1, 2, 3], dtype=torch.float32)
```

Najčešće korišteni atributi Tensora

```
x.shape  
x.dtype  
x.device
```



Napomena:

U stvarnim modelima Tensori najčešće dolaze iz Dataset-a i DataLoader-a, ali je važno razumjeti kako se stvaraju i što sadrže.

Tensor - osnovni objekt u PyTorchu

Device – CPU vs GPU

U PyTorchu se **svaki Tensor i model nalazi na određenom uređaju (*device*)**.

Najčešći uređaji:

- **CPU** → **cpu**
- **GPU (CUDA)** → **cuda**

Odabir device-a

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Premještanje Tensora na device

```
x = x.to(device)
```

Premještanje modela na device

```
model = model.to(device)
```

Tensor - osnovni objekt u PyTorchu

Device – CPU vs GPU

 **Važno (najčešća greška)**

Svi Tensori i model MORAJU biti na istom device-u.

 pogrešno:

- model na GPU
- podaci na CPU

 ispravno:

- model i podaci na istom uređaju

 **Ako PyTorch prijavi grešku vezanu uz device, prvo provjerite gdje se nalazi Tensor i model.**

PyTorch Tensor vs NumPy array

PyTorch Tensor vs NumPy array

PyTorch Tensor i NumPy array su **vrlo slični**, ali imaju različite uloge.

Sličnosti

- obje strukture predstavljaju višedimenzionalna numerička polja
- slične operacije (zbrajanje, množenje, indeksiranje)

Razlike

- **NumPy** radi isključivo na CPU-u
- **Tensor** može raditi na CPU-u ili GPU-u
- Tensor podržava **automatsko računanje gradijenata**

PyTorch Tensor vs NumPy array

Konverzija između NumPy i Tensora

```
import numpy as np  
import torch
```

```
a = np.array([1, 2, 3])  
x = torch.from_numpy(a)
```

```
b = x.numpy()
```



Napomena

- Tensor i NumPy array dijele istu memoriju
- promjena jednog može utjecati na drugi



NumPy se često koristi za obradu podataka, a PyTorch za treniranje modela.

Dataset i DataLoader

Dataset i DataLoader

Dataset – kako PyTorch predstavlja podatke

Dataset opisuje skup podataka iz kojeg PyTorch dohvaća uzorke.

- svaki uzorak = (ulaz, oznaka)
- Dataset **ne uči model**, samo isporučuje podatke
- PyTorch očekuje da Dataset implementira:
 - `__len__()`
 - `__getitem__(index)`

Zašto Dataset?

- omogućuje rad s velikim skupovima podataka
- podaci se učitavaju **po potrebi**, a ne svi odjednom
- odvaja **podatke** od **modela**



Model nikad direktno ne čita podatke — uvijek ide preko Dataset-a.

Dataset i DataLoader

Primjer Dataset-a (MNIST)

Uvoz

```
from torchvision import datasets, transforms
```

Kreiranje Dataset-a

```
transform = transforms.ToTensor()
```

```
dataset = datasets.MNIST(  
    root="./data",  
    train=True,  
    download=True,  
    transform=transform  
)
```

Dataset i DataLoader

Objašnjenje parametara

- root → gdje se podaci spremaju na disk
- train=True → treniramo ili testiramo
- download=True → automatsko preuzimanje ako ne postoji
- transform → obrada svakog uzorka prije vraćanja

Što Dataset vraća?

image, label = train_dataset[0]

- image → Tensor (npr. [1, 28, 28])
- label → klasa (npr. 7)



Dataset ne radi batchanje

Dataset vraća jedan uzorak po indeksu

Dataset i DataLoader

Podjela skupa podataka – random_split

PyTorch omogućuje podjelu Dataset-a na više dijelova.

```
from torch.utils.data import random_split
```

```
dataset_size = len(dataset)
```

```
train_size = int(0.9 * dataset_size)
```

```
val_size = dataset_size - train_size
```

```
train_dataset, val_dataset = random_split(  
    dataset,  
    [train_size, val_size]  
)
```

Dataset i DataLoader

DataLoader – organizacija podataka u batcheve

DataLoader uzima Dataset i organizira podatke za treniranje modela.

Dataset → daje jedan uzorak

DataLoader → daje batch uzoraka

Uvoz

```
from torch.utils.data import DataLoader
```

Kreiranje DataLoader-a

```
train_loader = DataLoader(  
    train_dataset,  
    batch_size=32,  
    shuffle=True  
)
```

Dataset i DataLoader

Objašnjenje parametara

- `train_dataset` → Dataset iz kojeg čitamo podatke
- `batch_size=32` → koliko uzoraka u jednom koraku
- `shuffle=True` → miješanje podataka prije svake epohe

Kako se koristi u praksi

```
for images, labels in train_loader:  
    print(images.shape)  
    break
```

Ako je `batch_size=32` za MNIST:

```
torch.Size([32, 1, 28, 28])
```

Transformacije

Transformacije

Transformacije – obrada podataka prije modela

Transformacije omogućuju da se svaki uzorak **automatski obradi** prije nego što dođe do modela.

Primjeri:

- pretvaranje slike u Tensor
- normalizacija
- promjena veličine slike
- augmentacija (rotacija, flip...)

Transformacije

Uvoz

```
from torchvision import transforms
```

Primjer transformacija

```
transform = transforms.Compose([  
    transforms.ToTensor(),  
    transforms.Normalize((0.5,), (0.5,))  
])
```

Što se ovdje događa?

- ToTensor()
 - pretvara sliku u PyTorch Tensor
 - skalira vrijednosti iz [0, 255] u [0, 1]
- Normalize(mean, std)
 - standardizira podatke
 - ubrzava i stabilizira treniranje

Transformacije

Kako se koristi u Dataset-u

```
train_dataset = datasets.MNIST(  
    root="./data",  
    train=True,  
    download=True,  
    transform=transform  
)
```



Transformacija se primjenjuje **na svaki uzorak automatski.**

Učitavanje gotove CNN mreže – ResNet18

Učitavanje gotove CNN mreže – ResNet18

PyTorch omogućuje učitavanje gotovih modela putem torchvision.models.

Uvoz

```
from torchvision.models import resnet18, ResNet18_Weights
```

Učitavanje modela s unaprijed naučenim težinama

```
weights = ResNet18_Weights.DEFAULT
```

```
model = resnet18(weights=weights)
```

Što znači weights=ResNet18_Weights.DEFAULT?

- model je već treniran (npr. na ImageNet datasetu)
- ima naučene konvolucijske značajke (feature extractore)
- možemo ga prilagoditi za novi zadatak (npr. MNIST)

DEFAULT označava preporučenu verziju težina za tu arhitekturu.

Učitavanje gotove CNN mreže – ResNet18

ResNet18 originalno ima **1000** klasa.
Za MNIST nam treba **10** izlaznih klasa.

→ Zato moramo promijeniti zadnji sloj.

Zamjena zadnjeg sloja (model.fc)

```
import torch.nn as nn
```

```
num_features = model.fc.in_features  
model.fc = nn.Linear(num_features, 10)
```

(Opcionalno) “Zamrzavanje” većine mreže

Ako želite trenirati samo novi zadnji sloj (brže, manje overfitta):

```
for param in model.parameters():  
    param.requires_grad = False
```

```
for param in model.fc.parameters():  
    param.requires_grad = True
```

Loss i Optimizer

Loss i Optimizer

Nakon definiranja modela, potrebno je definirati:

- funkciju gubitka (loss)
- optimizator

Loss funkcija

```
criterion = nn.CrossEntropyLoss()
```

Koristi se za klasifikaciju (višeklasni problem).

Optimizer

Ako treniramo samo zadnji sloj:

```
optimizer = torch.optim.Adam(model.fc.parameters(), lr=0.001)
```

Ako treniramo cijelu mrežu:

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```



Loss mjeri pogrešku

Optimizer ažurira parametre modela

Trening i Validacijska petlja

Trening i Validacijska petlja

Trening petlja (model uči)

U treningu model:

- računa izlaz (forward)
- računa gubitak (loss)
- računa gradijente (backward)
- ažurira parametre (optimizer step)

Osnovna struktura

```
model.train()
```

```
for images, labels in train_loader:  
    images = images.to(device)  
    labels = labels.to(device)
```

```
optimizer.zero_grad()
```

```
outputs = model(images)  
loss = criterion(outputs, labels)
```

```
loss.backward()  
optimizer.step()
```

Ključne napomene

- `model.train()` uključuje “train mode” (npr. Dropout/BatchNorm ponašanje)
- `zero_grad()` briše stare gradijente prije novog koraka
- `backward()` računa gradijente
- `step()` ažurira parametre

Trening i Validacijska petlja

Validacijska petlja (model ne uči)

U validaciji:

- model radi samo forward pass
- **ne računa gradijente**
- **ne ažurira parametre**

Osnovna struktura

model.eval()

with torch.no_grad():

```
for images, labels in val_loader:  
    images = images.to(device)  
    labels = labels.to(device)
```

```
outputs = model(images)  
loss = criterion(outputs, labels)
```

Ključne razlike u odnosu na trening

- model.eval()
 - isključuje Dropout
 - koristi stabilne BatchNorm vrijednosti
- torch.no_grad()
 - ne računa gradijente
 - smanjuje potrošnju memorije
 - ubrzava izvođenje

➡ Validacija služi za procjenu modela, ne za njegovo učenje.

**Spremanje modela na disk,
učitavanje modela s diska i
inferencija**

Spremanje modela na disk, učitavanje modela s diska i inferencija

Spremanje modela na disk

Standardni način u PyTorchu je spremiti **state_dict** (težine modela).

```
torch.save(model.state_dict(), "resnet18_mnist.pth")
```

Učitavanje modela s diska

Prvo kreiramo istu arhitekturu, zatim učitamo težine.

```
from torchvision import models  
import torch.nn as nn
```

```
model = models.resnet18(pretrained=False)  
model.fc = nn.Linear(model.fc.in_features, 10)
```

```
model.load_state_dict(torch.load("resnet18_mnist.pth", map_location=device))  
model = model.to(device)
```

Spremanje modela na disk, učitavanje modela s diska i inferencija

Inferencija (predikcija)

Kod inferencije:

- `eval()` mode
- `no_grad()`
- uzmemo klasu s najvećim score-om (`argmax`)

`model.eval()`

```
with torch.no_grad():  
    images, labels = next(iter(val_loader))  
    images = images.to(device)
```

```
outputs = model(images)  
preds = outputs.argmax(dim=1)
```

`preds` sadrži predviđene klase za batch.

Tensorboard u PyTorchu

Tensorboard u PyTorchu

Vizualizacija treninga – TensorBoard

TensorBoard omogućuje:

- praćenje loss funkcije
- praćenje accuracy metrike
- usporedbu eksperimenata
- vizualizaciju modela

PyTorch koristi:

```
from torch.utils.tensorboard import SummaryWriter
```

Tensorboard u PyTorchu

SummaryWriter -> Logiranje metrika

Kreiranje writer-a

```
from torch.utils.tensorboard import SummaryWriter  
  
writer = SummaryWriter("runs/resnet18_mnist")
```

Logiranje tijekom treninga

Unutar epohe:

```
writer.add_scalar("Loss/train", train_loss, epoch)  
writer.add_scalar("Accuracy/train", train_acc, epoch)  
  
writer.add_scalar("Loss/val", val_loss, epoch)  
writer.add_scalar("Accuracy/val", val_acc, epoch)
```

Na kraju

```
writer.close()
```

Tensorboard u PyTorchu

Instalacija TensorBoarda

TensorBoard nije automatski instaliran s PyTorchom.
Potrebno je instalirati dodatni paket:
`pip install tensorboard`

Pokretanje TensorBoarda

U terminalu:
`tensorboard --logdir=runs`

Zatim otvoriti u browseru:
`http://localhost:6006`

Pitanja ?

Slijedi rješavanje zadataka